NANYANG TECHNOLOGICAL UNIVERSITY

CCDS25-0878

INVESTIGATING THE USE OF INDUCTIVE LOGIC PROGRAMMING FOR KNOWLEDGE GRAPH COMPLETION

Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Science in Data Science and Artificial Intelligence
of the Nanyang Technological University

by

Ezra Koh

College of Computing and Data Science

March 2026

Abstract

This project investigates the application of logical rules in knowledge graph completion (KGC), specifically inductive relation prediction. Building on the LPRules model, the Soft margin Optimization with Flexible inference and Induced Augmentation (SOFIA) model introduces three new components. The soft margin component enforces ranking separation between correct and wrong predictions, while the flexible inference and induced augmentation components address the brittleness of logical rules, which is a primary weakness of rule-based models. The experimental results show that all three components improve LPRules performance across the board on three standard inductive KGC evaluation datasets, with induced augmentation standing out as the most effective. Notably, SOFIA outperforms TACT, a state-of-the-art graph neural network (GNN) inductive KGC method, on two out of the three datasets.

Acknowledgments

I am deeply thankful to Assistant Professor Timothy van Bremen for the opportunity to work on this project. His comments and suggestions have been instrumental to the success of the project and he has a very keen eye for stylistic improvements.

I am also grateful to my lovely girlfriend, Syarwina Ridwan, for reviewing my reports over multiple iterations and highlighting areas where clarification was needed.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

This chapter contains a brief introduction to knowledge graphs and the models used to solve its derivative tasks. A broad outline of the project scope is provided in the form of three research questions.

1.1 Background and Motivation

Knowledge graphs (KG) are datasets that contain facts about entities and how they relate to one another. Facts (or triples) in a KG can be represented in the format $relation(tail, head)$, e.g. $livesIn(DonaldTrump, WashingtonDC)$. A KG can be represented as a directed multigraph, which means that any two nodes (points) can be connected by several directed edges (lines), as two entities may be related in multiple ways, e.g. $worksIn(DonaldTrump, WashingtonDC)$.

Note that this triple format is consistent with the notation in LPRules (the model of interest), but is the reverse of the conventional $(head, relation, tail)$ and $relation(head, tail)$. For clarity, the graph traversal direction in LPRules and this project is $t \xrightarrow{r} h$. The difference is purely notational and does not change the graph representation of KGs. For example, LPRules calls *DonaldTrump* the tail whereas most other models call it the head, but in both graph representations, there is a directed arrow from *DonaldTrump* to *WashingtonDC*.

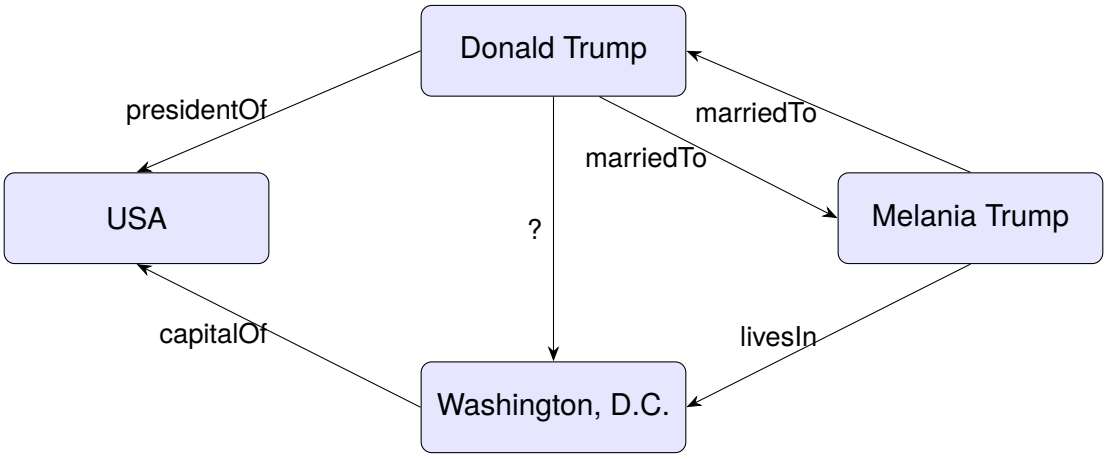


Figure 1.1: Example KG

Since graph algorithms can be applied directly to perform operations on KGs, they can be used efficiently in applications such as question answering, text generation, recommendation systems, natural language processing, financial forecasting, drug discovery, and medical diagnosis [1]. The relations and entities in KGs can be used to model any connections, like customers' interest in different products, semantic relations between words in a sentence, and interactions between proteins.

A small example KG is provided in Figure 1.1.

Since it is practically very difficult (or impossible in some cases) to gather all the information in any domain or subdomain, real-world knowledge graphs are often incomplete. The knowledge graph completion (KGC) machine learning task thus seeks to fill the blanks in existing KGs by inferring currently unknown facts based on existing facts.

KGC has three main areas: 1) triple classification, which is a binary classification task to estimate whether or not a triple is true, 2) entity prediction, where the model evaluates a test triple with a missing entity, i.e., $\mathit{relation}(\mathit{tail}, ?)$ or $\mathit{relation}(?, \mathit{head})$, and 3) relation prediction, where the test triple has a missing relation, i.e., $?(\mathit{tail}, \mathit{head})$. In the evaluation phase of entity/relation prediction, the model uses its trained heuristic to rank each entity/relation by how suitable it is to fill in the blank.

Definition 1 *A relation prediction model takes a masked test triple ?(tail,head) as input and returns scores for all candidate relations as output.*

KGC models can be trained and evaluated in either the transductive or inductive setting [2]. In the transductive setting, all entities in the test set are present in the training set. This allows the graph constructed using facts in the training set to be used when ranking entities/relations for test facts. In the inductive setting, the entities in the training set and the entities in the test set are disjoint, meaning that they share no common entities. This means that the training graph cannot be used in the evaluation phase, and another graph must be constructed using only the facts in the test set.

In the transductive setting, it is assumed that the set of entities in a KG is fixed. This assumption does not hold in many practical use cases. In many real-world KGs, more entities are added over time, e.g. new users and products on e-commerce platforms. The ability to make predictions on these new entities without constantly incurring the high costs of model training is essential [3].

1.2 Types of Models and Interpretability

Relation prediction models can generally be classified into four types: rule-based, representation learning, deep learning, and large language model-based [1]. Of these, rule-based models possess an inherent advantage in inductive KGC [4] and are the only interpretable models.

Definition 2 *A model is interpretable if its reasoning mechanisms make sense to a human observer [5].*

Whereas the three other types of models make predictions by performing many complex vector calculations and introducing black-boxes, which are inherently uninterpretable [6], rule-based models leverage the graph properties of KGs to infer missing triples by learning path rules, e.g.

$$\texttt{marriedTo}(A, B) \coloneq \texttt{marriedTo}(B, A) \tag{1.1}$$

$$\texttt{livesIn}(A, C) \coloneq \texttt{marriedTo}(A, B) \wedge \texttt{livesIn}(B, C) \tag{1.2}$$

$$\texttt{capitalOf}(A, C) \coloneq \texttt{livesIn}(B, A) \wedge \texttt{presidentOf}(B, C) \tag{1.3}$$

which can be written in plain English as

1. Marriage is a two-way street (i.e. a symmetric relation)
2. Married people live together
3. The president of a country lives in the capital of that country

1.3 Objectives and Project Scope

This project aims to advance the study of inductive relation prediction models by adapting LPRules [7] for the relation prediction task. LPRules is a rule-based model that uses linear programming to learn optimal rule weights, and was originally proposed for the entity prediction task. The Soft margin Optimization with Flexibility and Induced Augmentation (SOFIA) model contains 3 enhancements to LPRules, namely 1) *Soft margin*, 2) *Induced augmentation*, and 3) *Relaxed rules* (see Section 3.2).

The hypothesis tested in this project is that a rule-based model is capable of achieving state-of-the-art performance in inductive relation prediction, which means that explainability does not have to be sacrificed for performance. Four research questions are proposed and answered in this study.

RQ1: Can the LPRules model be adapted for relation prediction, and how does it compare to current relation prediction models?

RQ2: Does replacing the LPRules penalty with a soft margin penalty improve performance?

RQ3: Do induced augmentation and relaxed rules add noise to predictions, or are they able to preserve the discriminative power of learned rules?

RQ4: Are incomplete rule paths caused by missing intermediate facts better handled by induced augmentation or relaxed rules?

Chapter 2

Related Work

In this chapter, past literature on knowledge graph completion models is explored. Some inspiring literature relevant to the contributions of this project are also discussed.

2.1 Types of Models

2.1.1 Representation Learning

The most commonly used type of model in relation prediction is representation learning, which is the process of translating entities and relations to a low-dimensional embedding space [1]. Representing entities and relations as vectors provides a mathematically clean approach to estimating the connections between them and inferring missing triples. However, this approach has two main weaknesses: interpretability and generalization.

Translation-based embeddings treat relations as transformations between entities [1]. In this method, embeddings for each entity and relations are learned such that for all triples $r(h,t)$, $hr \approx t$. A notable example of translation-based embedding models is TransE [8] and its variants. TransE uses geometric relations for predictions, which are slightly more interpretable but struggle with complex relations [1]. TransH overcomes this by mapping each relation to a hyperplane, while TransR learns projection matrices to map entities from the entity vector space to an independent vector space for each relation.

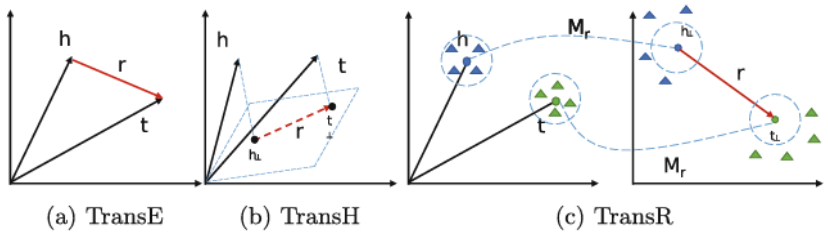


Fig. 1. Simple illustrations of TransE, H, R model

Figure 2.1: Illustration of TransE, H, R [1]

Semantic-matching embeddings represent semantic aspects of entities and relations as vectors and use similarity score functions to measure the plausibility of facts. Both translational and semantic embeddings are uninterpretable. This is shown by the amount of research devoted to explaining embeddings and making them human-readable [9, 10]. They also cannot be used to generalize, as they map each word to a specific vector.

In the context of relation prediction, these models are unable to predict relations between entities they have not seen during training. By definition, this excludes most representation learning models from inductive KGC. GraphSAGE [11] is a representation learning model trained for inductive learning, but it relies on the presence of node features, which are not provided in many KGs [1].

2.1.2 Large Language Models

Large language models (LLMs) such as BERT have been used to great success in KGC tasks, particularly in low-data resource scenarios [1]. This is because the training process of these LLMs essentially provides them with the query-friendly properties of KGs.

KG-BERT [12] reframes facts in the KG as sentences to use as input, and fine-tunes a pre-trained BERT model to compute plausibility scores of triples. KG-BERT achieves strong performance in all three KGC tasks by making effective use of the semantic relatedness and contextual information that exists in a KG. However, it suffers from a long runtime and low interpretability (attention cannot be an explanation [13]).

2.1.3 Graph Neural Networks

Graph neural networks (GNNs), which are a type of deep learning model, also show great potential, especially in the inductive setting. One significant limitation of GNNs is that they struggle to model multiple relations between a pair of entities (see Section 1.1) [1]. The reasoning structure of GNNs is also clearly uninterpretable.

GraIL [3] uses the graph structure information of a KG, applying the message passing mechanism of GNNs to score a triple given its neighbouring entities and relations. TACT [4] expands on GraIL by using the topological structure of a KG to model semantic correlations between its relations. Seven topological patterns are used to describe the correlations between any two relations.

As illustrated in Figure 2.2 [4], these are head-to-tail (H-T), tail to-tail (T-T), head-to-head (H-H), tail-to-head (T-H), parallel (PARA), loop (LOOP) and not connected (NC). The KG is then converted into a relational correlation graph (RCG), in which nodes represent relations and edges represent the topological patterns between them. The degrees of influence that are exerted by the topological structures on any two relations are learned using attention networks.

2.1.4 Rule-Based Models

Rule-based models perform inference using logical rules, which are easy for experts and stakeholders to supervise, verify, and edit.

The Markov logic network (MLN)[14] elegantly combines probabilistic graphical models and first-order logic in a single representation. This is a foundational development in KGC tasks which handles uncertainty and imperfect knowledge efficiently by learning probabilistic weights for logical rules.

Neural LP [15] uses neural networks to learn the structure (which rules to use) and parameters (confidence associated with each rule) for a KG. It achieved state-of-the-art results at publication and, despite being almost a decade old, continues to be cited as a baseline in many KGC research studies.

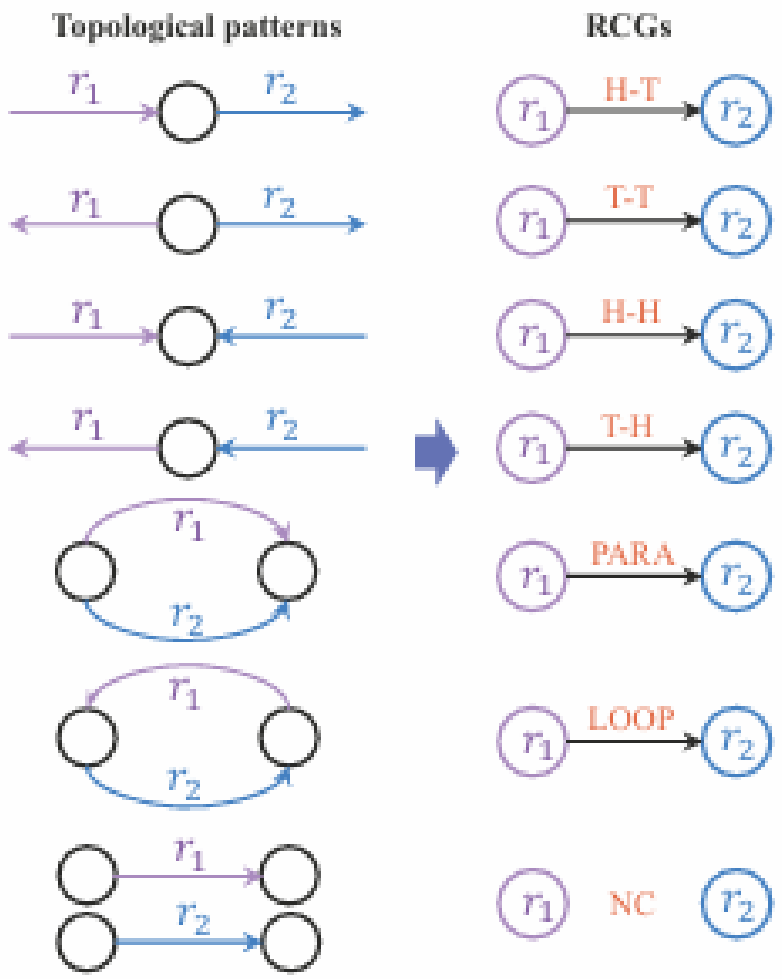


Figure 2.2: Types of topological patterns [4]

LPRules [7] is the model of interest for this project. It uses linear programming (LP) to learn weights for each rule and was originally designed for the entity prediction task.

Rule-based models are limited in their expressive ability [1] because of their inflexible structure. Logical rules are also very brittle. If one link in the rule path is broken, the rule is not satisfied. Given the incompleteness of knowledge graphs, it is likely that the very facts needed to satisfy a rule that correctly predicts a missing fact are themselves missing from the dataset! [16]

Therefore, this project proposes some modifications to overcome the weaknesses of rule-based models. The next section lists some literature which inspired some concepts in the modifications.

2.2 Inspirations for Modifications

2.2.1 Soft Margin SVM

The soft margin support vector machine (SVM) can be directly generalized to ranking problems using the fixed margin policy [17]. This approach maximises the margin between the closest pair of classes.

In recent work, SVM has been applied to ranking problems in diverse fields such as education analytics (ranking individual student skills and skill prerequisites for courses) [18] and medical diagnosis (ranking severity of dysarthria in Parkinson's disease patients) [19].

Since the goal of relation prediction models is to assign the highest score to the correct relation over all other competing relations, relation prediction can be formulated as a ranking problem. The soft margin concept from SVM can thus be applied to KGC. In rule-based models, rules that consistently predict correct relations receive high weights and are like support vectors lying on the margin, perfectly seperating classes. Rules with occasional mistakes are downweighted by applying slack penalties.

2.2.2 Forward Chaining

Unified framework for combining Knowledge graph Embedding (KGE) with logical Rules (UniKER) [16] proposed the application of forward chaining (see Section 3.2.2) to augment KGC training data.

In the UniKER model, logical rules and KGE work together iteratively. Missing facts are inferred and added to the training using a small set of logical rules. A KGE model is then used to identify potentially useful triples and classify them as true or false, and to identify potentially incorrect triples and remove them from the training KG. This process is repeated for 2 to 4 iterations.

No prior work has investigated the application of forward chaining in the inference stage of KGC.

Chapter 3

Model

The LPRules model was selected as the base model for this project because it performs reasonably well on entity prediction and has a relatively short training time. Three modifications to the LPRules model are proposed and implemented in the Soft margin Optimization with Flexible inference and Induced Augmentation (SOFIA) model.

3.1 LPRules

LPRules [7] is a rule-based model that generates path rules and uses linear programming (LP) to learn weights for each rule. During the testing phase, when the LPRules model sees the test triple $R(T, ?)$ or $R(?, H)$, it iterates over each rule learned for relation R and attempts to follow the corresponding path. All entities that satisfy the path for a given rule receive a score equal to the weight of that rule. After all the rules for relation R have been checked, the correct entity is ranked by the sum of the scores it received, compared to all other entities.

The model flow of LPRules is depicted in Figure 3.1.

Path rules are generated in a two-step process. First, all rules of length one and two are iteratively generated, and those that successfully cover (i.e., are satisfied by) a large proportion of training facts are added. Longer rules are then added by identifying facts $R(X, Y)$ associated with rules that may reduce the LP solution value (z_{min}) and finding

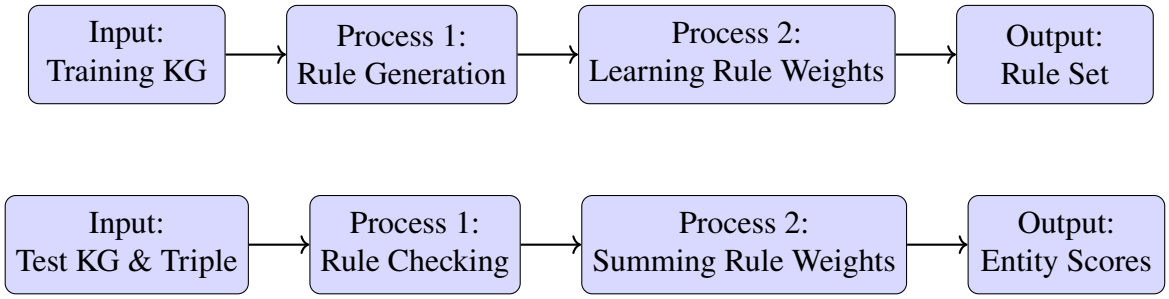


Figure 3.1: Model flow for training and testing

the shortest path between X and Y.

The objective function of the LP for learning rule weights is

$$z_{min} = \min \sum_{i=1}^{m} \eta_i + \tau \sum_{k \in K} neg_k w_k \tag{3.1}$$

subject to the following constraints

$$\forall i \in E_r, \sum_{k \in K} a_{ik} w_k + \eta_i \geq 1 \tag{3.2}$$

$$\sum_{k \in K} (1 + |C_k|) w_k \leq \kappa \tag{3.3}$$

$$\forall k \in K, w_k \in [0, 1] \tag{3.4}$$

$$\forall i \in E_r, \eta_i \geq 0 \tag{3.5}$$

Symbol	Description
η_i	penalty on false negatives
τ	tradeoff between specificity and sensitivity
neg_k	number of false positives
w_k	weight of rule k
E_r	set of triples with relation r
K	set of rules
a_{ik}	binary indicator that a path satisfying rule k connects from X_i to Y_i
$\|C_k\|$	length of rule k

Table 3.1: LP notations

The LP solution value z_{min} is equal to the lowest possible sum of penalties given to the model for false negatives and false positives. The first constraint imposes a penalty η

when rules do not give the full score of 1 to training facts. This is similar to penalizing a model for false negatives, which trains it to give higher weights to rules that cover many facts. The objective function tries to minimize the sum of these penalties and the total score of negative triples, which can be thought of as penalizing for false positives, which helps it to learn lower weights for rules that cover too many negative triples (i.e. wrong facts).

A negative triple $r(t', h)$ or $r(t, h')$ is a "fact" generated by taking a training fact $r(t, h)$ and replacing one of its entities with another entity selected at random. neg_k is the number of negative triples which satisfied $rule_k$. The parameter τ thus balances the tradeoff between assigning high scores to correct facts and low scores to wrong facts. The second constraint κ limits the complexity of the model, which is defined as the number of rules plus their total length. The third and fourth constraints simply define the bounds for rule weights (w_k) and penalties (η_i). The depth-first search algorithm is used to check satisfaction of rule paths. Note that, unlike many rule-based models, rule weights in LPRules do not represent probabilities.

The LP is set up once for each relation. The size of the LP is dependent on the number of triples with that relation ($|E_r|$) and the number of rules generated ($|K|$). It has $|E_r| + |K|$ variables and $|E_r| + 1$ constraints.

LPRules is able to approach state-of-the-art performance in inductive entity prediction, even when compared to much more complex models. It is thus hypothesised that a model based on LPRules, with a few additions, will be able to achieve similar results in inductive relation prediction. The models used for comparison are NBFNet+CP and RED-GNN+CP, which were proposed in a recent paper by Zhixiang Su, a PhD candidate from Nanyang Technological University. The metrics reported here are taken from their respective papers [7, 20].

The main results published in the LPRules paper were verified by running the model using a local WSL terminal and all the parameters defined by the authors. The ILOG CPLEX Studio from https://skillsbuild.org/college-students was used to compute the LP solutions. This took up to 6 times as long as the reported runtime due to hardware differences. The approximate runtimes for each dataset were, respectively, YAGO3: 36

Algorithm 1 LPRules

Input: Train, test, & validation datasets
Parameters: Sets of τ values & complexity bounds κ
Output: Rules & evaluation metrics
for each relation r in train **do**
    Call Rule Heur. 1 & 2 to generate initial rule set $\mathcal{K}_0$
    Set up LPR_0 from $\mathcal{K}_0$ using min τ & min κ and solve
    bestMRR $\leftarrow 0$, best$\tau \leftarrow -\infty$
    for $i \leftarrow 1$ to MaxIter **do**
        Generate new rules with modified Rule Heur. 2
        Add new rules with reduced cost < 0 to $\mathcal{K}_{i-1}$
            to form $\mathcal{K}_i$, set up LPR_i from $\mathcal{K}_i$ and solve
    end for
    for each τ **do**
        for each κ **do**
            Set up LPR_i from $\mathcal{K}_i$ using current τ & κ and solve
            Use solution & compute MRR on validation facts
            if MRR > bestMRR **then**
                bestMRR $\leftarrow$ MRR
                best$\tau \leftarrow \tau$, best$\kappa \leftarrow \kappa$
            end if
        end for
    end for
    Set up LPR_i from $\mathcal{K}_i$ using bestτ, bestκ and solve
    Output rules with corresponding weights in LPR_i
end for
Compute evaluation metrics on test set

	WN18RR				FB15k-237	
	V1	V2	V3	V4	V1	V2
MRR						
LPRules	0.682	0.646	0.398	0.609	0.313	0.396
NBFNet+CP	0.702	0.658	0.393	0.605	0.387	0.423
RED-GNN+CP	0.708	0.694	0.425	0.648	0.383	0.463
Hits@1 (%)						
LPRules	63.6	60.9	35.9	57.7	27.1	31.5
NBFNet+CP	62.2	57.8	32.9	53.7	28.5	28.3
RED-GNN+CP	66.0	63.7	36.5	60.3	31.6	36.8

Table 3.2: Inductive entity prediction results

hours, FB15k-237: 24 hours, WN18RR: 1 hour, Kinship: 3 minutes, UMLS: 1 minute. All metrics were very close to reported (most differences <0.01, some <0.03), except Hits@10 on YAGO3, which was slightly lower than reported (0.6 vs 0.684).

The LPRules code (https://github.com/IBM/LPRules/tree/main) was then edited to perform relation prediction instead of entity prediction during the test phase. At this stage, no changes were made to the LP or scoring methods.

One significant change that had to be made to the code was the parallel computing implementation. The original LPRules code used parallel computing in both training and testing by sorting each triple into a different group based on its relation r and sending each group to a different thread. This works because the LPRules model was applied to the entity prediction task. When ranking entities for a test triple $r(t, ?)$, rules of relations other than r are not needed to compute the scores for all entities. However, this does not apply to the relation prediction task. If the same parallel computing method is applied, only the correct relation receives a positive score, since only those rules are available in that thread. The solution is simple: the model is trained in parallel threads, rules for all relations are written to a single text file, and this entire rule set is used to evaluate the model in parallel threads.

In the training stage, the soft margin constraint is added to improve ranking seperation. Induced augmentation is conducted pre-inference, and the relaxed rule format (flexible inference) is introduced at the inference stage. Both of these are methods to overcome the brittleness of path rules and enable rules to be satisfied more frequently.

Model performance is limited by rule coverage: if none of the rules for the correct relation are satisfied, it is unlikely to get a good rank (under a random tie-breaking system).

Definition 3 *Coverage is the proportion of test triples for which at least one rule belonging to the correct relation was satisfied.*

To overcome this limit, the model is provided with induced facts to enable more rule paths to be satisfied, and partial scores are awarded when partial paths are satisfied.

3.2 SOFIA

3.2.1 Soft Margin

The soft margin concept is borrowed from SVMs [21]. This LP modification increases rule weights when the correct relation scores above all other relations by a certain margin (γ), effectively rewarding ranking separation. The model flow is mostly the same as in 3.1, with the exception of outputting Relation Scores instead of Entity Scores.

The new objective function of the LP is

$$z_{min} = \min \sum_{i=1}^{m} \eta_i + \lambda \sum_{i=1}^{m} \xi_i \tag{3.6}$$

subject to the additional constraint

$$\forall i \in E_r, \sum_{k \in K} (a_{ik} - s_k) w_k + \xi_i \geq \gamma \tag{3.7}$$

where s_k is the proportion of negative triples $r'(t, h)$ (i.e. incorrect facts generated by randomly replacing the relation of a known fact with another relation) that have a path satisfying $rule_k$. The original constraints are all unchanged.

The soft margin LP is larger than the original LP model, with $2|E_r| + |K|$ variables and $2|E_r| + 1$ constraints.

3.2.2 Induced Augmentation

Definition 4 *The forward chaining process starts with a set of facts and repeatedly applies the given rules to generate new facts [22].*

The idea of augmenting the test graph to enable path-finding is drawn from the concept of forward chaining. The induced fact set is generated by applying the learned rules to all tail-head combinations of all entities in the test set. Each tail-head combination and its highest-scoring relation are added to the induced fact set if the score of that relation

is at least 1 (the maximum score awarded for satisfying one rule). This ensures the quality of the induced fact set and avoids inducing facts on demand, which is likely to involve redundant inductions in the parallel computing implementation. If two or more relations tie for the highest score, all of them will be added to the induced fact set.

Algorithm 2 Induced augmentation

```
for each entity e_t in test do
    for each entity e_h in test do
        if e_t = e_h then
            continue
        end if
        bestRelations = ∅
        maxScore = 1
        for each relation r in test do
            score = getScore(e_t, r, e_h)
            if score > maxScore then
                maxScore = score
                bestRelations = {r}
            else if score = maxScore then
                bestRelations = bestRelations ∪ {r}
            end if
        end for
        for each r in bestRelations do
            write induced fact (e_t, r, e_h) to output file
        end for
    end for
end for
```

The induced fact set is not used to induce more facts (contrary to the standard definition of forward chaining), as further iterations require exponentially more computing time and are more likely to introduce noisy facts. Note that the induced facts are only added to the test graph for the purpose of path-finding and are not used as test triples, i.e. they are excluded from the evaluation metrics.

For an example of how induced augmentation enables better predictions, see Figure 1.1. Consider the case where the model attempts to predict the relation of the test triple ?(*WashingtonDC*, *USA*) using the three example rules. Since Rule 1.3 is the only rule of the correct relation *capitalOf*, it has to be satisfied to make the correct prediction. This means that two facts are required to be known in the KG: *livesIn*(*DonaldTrump*, *WashingtonDC*) and *presidentOf*(*DonaldTrump*, *USA*). But the

former is missing from the KG, so Rule 1.3 does not fire. Induced augmentation resolves this by first using Rule 1.2 to induce the missing fact, thus creating a path that satisfies Rule 1.3 and allowing the correct relation *capitalOf* to be predicted.

Despite taking precautions such as not inducing facts repeatedly, and requiring the induced facts to have a minimum score of 1, the induced fact set is still likely to contain some noisy facts. There are two kinds of noisy facts: incorrect facts, e.g. $capitalOf(NewYork, USA)$ and implausible facts, e.g. $presidentOf(NewYork, USA)$.

Definition 5 *An implausible fact is a fact which contains entities which are inconsistent with the entity types corresponding to its relation.*

Implausible facts, being the random nonsense that they are, rarely appear as links in logically consistent rule paths as plausible facts would not connect to them. Incorrect facts, on the other hand, are logically consistent and thus able to affect rule path satisfaction. However, the negative effects are likely to be largely or completely mitigated by the sheer increase in satisfied rule paths.

The level of competition that the predicted relation has to outperform in order to achieve the highest score is greatly elevated by the new paths created by induced facts. If the highest scoring relation achieved its score by satisfying five different rule paths, each containing one induced fact, this is more convincing than a relation that only satisfies one or two rule paths using zero induced facts. Furthermore, under the open-world assumption inherent in KG incompleteness, manually identifying and removing noisy facts is non-trivial and too time-consuming. For these reasons, no attempts are made to remove noisy facts.

3.2.3 Relaxed Rules

Definition 6 *A relaxed rule path is a truncation of the original rule path, which only includes the first and last hops.*

For example, the rule

$$R(A, D) :- P(A, B) \wedge Q(B, C) \wedge S(C, D) \tag{3.8}$$

is truncated to the relaxed rule

$$R(A, D) :- P(A, B) \wedge S(C, D) \tag{3.9}$$

where B and C can be any two entities. Satisfying at least the first and last hops is the maximum relaxation, as they preserve anchoring to the test query entities A and D. During the test phase, if a path satisfying a given rule cannot be found, the corresponding relaxed rule path is searched instead.

Relaxed rules are worth $\frac{bw_k}{2^d}$ of their original rule weight, where $d = ruleLength - 2$, i.e. the number of relations in the rule that were dropped, and b is a user-defined parameter in the range $(0, 1]$. The reasons for this weight reduction factor are 1) to not allow relaxed rule scores to dominate full rules, and 2) to apply more penalty for relaxing longer rules, as it leads to more evidence loss than relaxing shorter rules. By definition, relaxed rules are only computed for rules of length 3 or more.

When relaxed rules interact with induced facts, two implausible or incorrect induced facts could by themselves satisfy a relaxed rule path. Relaxed rules are meant to provide partial credit for incomplete evidence in the KG and thus are not allowed to fire on induced facts. Since induced facts themselves are predictions and not evidence, allowing relaxed rules to fire on them is likely to compound uncertainty in predictions.

3.2.4 Minor modifications

Two minor modifications - frequency bias and specificity-weighted rules were also proposed and implemented, but did not significantly affect model performance.

Frequency bias for each relation r is computed as $b_r = \frac{\#r(?,?)}{\#triples}$, i.e. the proportion of triples containing relation r. The bias is scaled to the range $[0.001, 0.01]$, which is less than the lowest rule weight. The bias is then added to the scores of each relation during the test phase. It can also be observed by the LP during training. It replaces randomness as a tiebreaker. The reasoning is that out of two or more relations that have the same score, the relation that occurs most in the training set is likely to occur most in the test set and thus most likely to be the correct relation.

The specificity of a rule is defined here as the proportion of negative triples that do not satisfy it. Specificity can be incorporated into rule weights by multiplying w_k with $(1 - s_k)$ (see Section 3.2.1).

Chapter 4

Experimental Setup and Results

This chapter presents the practical details and results of the experiments conducted in this project and compares them with the results of TACT [4] on the relation prediction task.

4.1 Machine and Software Details

Following LPRules [7], the software used for solving LPs is the academic version of IBM ILOG CPLEX Optimization Studio 22.1.1 from https://skillsbuild.org/college-students. All experiments were conducted on a WSL terminal with an Intel(R)Core(TM) i5-5300U CPU @ 2.30GHz. The IBM solver does not use GPUs as they are less efficient than CPUs for solving linear programming problems.

4.2 Datasets and Parameters

All datasets used (WN18RR, FB15k-237, NELL-995: v1-v4) were constructed by [3] and downloaded from https://github.com/kkteru/grail. These datasets are standard for evaluating KGC models in the inductive setting.

The original LP model parameters were kept the same as in LPRules. The maximum rule length was 6 for WN18RR and 4 for FB15k-237 and NELL-995. WN18RR is allowed to have longer rules because it has fewer relations than the other two datasets.

The set of possible rules expands exponentially as rule length increases; the terminal ran out of memory when the maximum rule length for NELL-995 was increased to 5.

When the induced fact set is very large, facts are randomly sampled from it to prevent out-of-memory errors. In this project, the maximum number of induced facts added to the test graph is set to 200,000, which affects FB15k-237 v3 and v4 and NELL-995 v3.

The enhanced LP model parameters used in the experiments are recorded in Table 4.1

Parameter	Value
γ	0.1
λ	0.01
b	0.1

Table 4.1: Enhanced LP model parameters

	#Relations	#Entities	#Triples	#Induced Facts
		WN18RR		
Ind-V1	9 (8)	2,746 (922)	6,678 (1,991)	16,440
Ind-V2	10 (10)	6,954 (2,757)	18,968 (4,863)	33,416
Ind-V3	11 (11)	12,078 (5,084)	32,150 (7,470)	73,618
Ind-V4	9 (9)	3,861 (7,084)	9,842 (15,157)	180,114
		FB15k-237		
Ind-V1	180 (142)	1,594 (1,093)	5,226 (2,404)	11,660
Ind-V2	200 (172)	2,608 (1,660)	12,085 (5,092)	84,649
Ind-V3	215 (183)	3,668 (2,501)	22,394 (9,137)	945,918
Ind-V4	219 (200)	4,707 (3,051)	33,916 (14,554)	808,410
		NELL-995		
Ind-V1	14 (14)	3,103 (225)	5,540 (1,034)	50,952
Ind-V2	88 (79)	2,564 (2,086)	10,109 (5,521)	65,516
Ind-V3	142 (122)	4,047 (3,566)	20,117 (9,668)	244,819
Ind-V4	77 (61)	2,092 (2,795)	9,289 (8,520)	106,589

Table 4.2: Dataset statistics

4.3 Metrics

The main performance metrics for KGC models, are mean rank (MR), mean reciprocal rank (MRR), and Hits@1. They are all based on the predicted ranks of the ground truth.

Given the *i*th test triple, $rank_i$ is computed by calculating the scores of all relations and sorting them from highest to lowest, such that the highest scoring relation has rank 1 and the lowest scoring relation has rank #Relations. Then $rank_i$ is simply the rank of the correct relation in this ordered list.

Let n be the number of test facts. Mean rank (MR) is the average of the predicted ranks. A smaller MR value suggests better performance.

$$MR = \frac{1}{n}\sum_{i=1}^{n} rank_i \tag{4.1}$$

While it is simple to calculate, MR is sensitive to outliers and thus prone to exploding to high values. Thus, the performance of the proposed models and models for comparison is evaluated on MRR and Hits@1, and MR is ignored.

$$MRR = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{rank_i} \tag{4.2}$$

and

$$Hits@1 = \frac{1}{n}\sum_{i=1}^{n} I(rank_i), I(x) = \begin{cases} 1 & \text{if } x = 1 \\ 0 & \text{if } x > 1 \end{cases} \tag{4.3}$$

Both MRR and Hits@1 can have values ranging from 0 to 1, and a higher value indicates better performance.

For an example of how MR is less reliable than MRR: Model 1 and Model 2 rank the correct relations on 4 test facts as [1,1,2,100] and [10,10,10,10] respectively. Then

$$MR_1 = 26 > 10 = MR_2 \tag{4.4}$$

so MR suggests that Model 2 is better than Model 1. However, Model 1 is clearly preferable, and MRR accurately demonstrates this.

$$MRR_1 = \frac{1}{4}(1 + 1 + \frac{1}{2} + \frac{1}{100}) = 0.6275 > 0.1 = MRR_2 \tag{4.5}$$

4.4 Results

The results of the experiments are compared with the results of GraIL and TACT reported in [4]. MRR and Hits@1 are computed in the filtered setting following [23]. The best metrics for each dataset are marked in bold.

Answering **RQ1**, LPRules can be adapted for relation prediction, but its performance falls short of TACT on many datasets. SOFIA achieves state-of-the-art performance on FB15k-237 and NELL-995, but lags behind on WN18RR.

	WN18RR				FB15k-237				NELL-995			
	V1	V2	V3	V4	V1	V2	V3	V4	V1	V2	V3	V4
MRR												
GraIL	0.851	0.772	0.549	0.746	0.054	0.060	0.060	0.056	0.337	0.104	0.058	0.072
TACT	**0.995**	**0.984**	**0.895**	**0.988**	**0.830**	0.815	0.752	0.575	**0.880**	0.680	0.548	0.571
LPRules	0.669	0.637	0.489	0.656	0.327	0.459	0.482	0.512	0.715	0.394	0.440	0.153
SOFIA	0.917	0.848	0.736	0.945	0.744	**0.851**	**0.771**	**0.783**	0.803	**0.746**	**0.773**	**0.837**
Hits@1 (%)												
GraIL	74.9	62.8	54.9	61.0	1.6	1.3	0.3	1.7	14.6	1.0	0.7	1.7
TACT	**99.5**	**97.8**	**85.2**	**98.2**	**74.1**	71.7	60.9	37.8	**77.6**	53.3	35.4	44.4
LPRules	46.4	45.5	32.8	47.1	21.0	28.1	31.1	32.9	45.2	32.7	34.4	8.1
SOFIA	84.9	77.9	65.5	91.0	70.3	**79.0**	**69.8**	**70.2**	61.7	**68.9**	**72.1**	**78.8**

Table 4.3: Inductive relation prediction results

4.5 Ablation Study

Ablation studies are conducted to measure the contribution of each component in the SOFIA model. SOF is the SOFIA model without induced augmentation, SOIA is the SOFIA model without relaxed rules, and SO is the SOFIA model without either. The metrics for the minor enhancements are not reported as they do not contribute significantly to model performance. The SO results answer **RQ2** positively; replacing the LPRules penalty with a soft margin penalty does improve performance.

Table 4.4 shows that induced augmentation (SOIA) has a greater effect on the model performance across all datasets, compared to relaxed rules (SOF). It is also observed that the performance improvement of SOF is much greater on WN18RR than it is on FB15k-237 and NELL-995. This is likely due to the hierarchical structure of WN18RR. In the paths of rules learned for WN18RR, the first and last hops are highly informative, while the intermediate hops are just natural progressions up or down the hierarchical tree. Since FB15k-237 and NELL-995 cover multiple domains, the intermediate hops apply significant conditioning, and thus relaxed rules are not as helpful. Relaxed rules improve coverage by enabling partial path matches without the same competitive pressure applied by the new complete paths that can be found with induced facts.

It is observed that there are some datasets on which SOIA is better, and some on which SOFIA is better. SOIA beats SOFIA on average Hits@1 (74.44 against 74.18), and SOFIA wins by a small margin on average MRR (0.8128 against 0.8125). This suggests that SOIA and SOFIA have similar explanatory power, thus by Occam's razor, the SOIA model is generally preferable. From all these observations, the answer to **RQ4** is that expanding the KG with induced augmentation is a more effective solution to incomplete paths than allowing the model to take shortcuts with relaxed rules.

Definition 7 *Occam's razor states that when two or more hypotheses have equal explanatory power, the hypothesis that requires the fewest assumptions should be preferred.*

	WN18RR				FB15k-237				NELL-995			
	V1	V2	V3	V4	V1	V2	V3	V4	V1	V2	V3	V4
MRR												
SO	0.737	0.656	0.502	0.695	0.359	0.539	0.577	0.614	0.789	0.462	0.467	0.174
SOF	0.831	0.753	0.636	0.855	0.378	0.553	0.587	0.626	0.727	0.471	0.515	0.191
SOIA	0.927	0.856	0.726	0.938	0.737	0.849	0.767	0.778	0.819	0.743	0.771	0.839
SOFIA	0.917	0.848	0.736	0.945	0.744	0.851	0.771	0.783	0.803	0.746	0.773	0.837
Hits@1 (%)												
SO	57.7	49.7	35.2	53.7	28.9	47.4	49.3	52.9	59.5	42.1	37.6	10.9
SOF	74.8	66.7	54.6	77.5	33.0	48.6	50.1	53.8	47.2	42.8	42.5	12.2
SOIA	86.8	79.3	63.5	90.3	69.6	78.9	69.5	69.7	66.3	68.3	71.9	79.2
SOFIA	84.9	77.9	65.5	91.0	70.3	79.0	69.8	70.2	61.7	68.9	72.1	78.8

Table 4.4: Ablation study metrics

Chapter 5

Discussion

The experimental results are analysed and discussed in this chapter.

5.1 Induced augmentation

Since the induced augmentation component is found to have a much more significant contribution to SOFIA, this section explores its impact in detail.

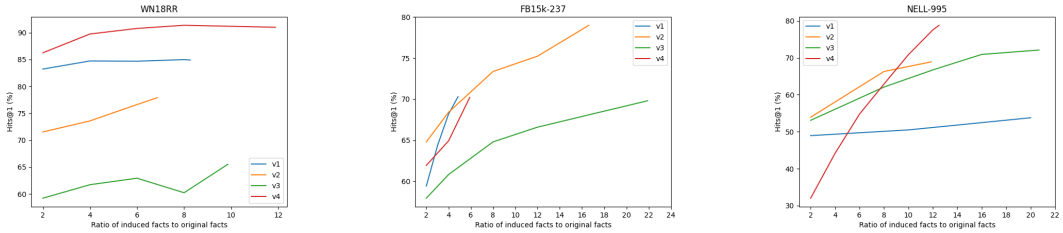
5.1.1 Interaction of Induced Facts and Relaxed Rules

The effect of allowing relaxed rules to fire on induced facts was investigated. This configuration is denoted as SOFIA+ in Table 5.1, which shows that SOFIA+ underperforms on all datasets except NELL-995 v1. The results confirm that blocking relaxed rules from firing on induced facts is the correct decision.

	WN18RR				FB15k-237				NELL-995			
	V1	V2	V3	V4	V1	V2	V3	V4	V1	V2	V3	V4
MRR												
SOFIA	0.917	0.848	0.736	0.945	0.744	0.851	0.771	0.783	0.803	0.746	0.773	0.837
SOFIA+	0.908	0.841	0.728	0.940	0.743	0.849	0.767	0.780	0.872	0.740	0.772	0.823
Hits@1 (%)												
SOFIA	84.9	77.9	65.5	91.0	70.3	79.0	69.8	70.2	61.7	68.9	72.1	78.8
SOFIA+	84.4	77.4	64.8	90.4	70.0	78.7	69.4	69.8	77.7	68.0	72.0	76.7

Table 5.1: Interaction between relaxed rules and induced augmentation

5.1.2 Size of Induced Fact Set



(a) WN18RR (b) FB15k-237 (c) NELL-995

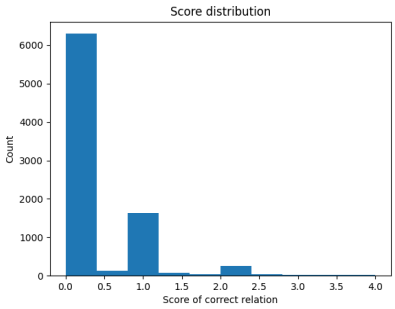
Figure 5.1: Hits@1 trends upward with more induced facts

The effect of the number of induced facts added for path-finding is investigated by randomly sampling induced facts in varying proportions of the size of the original test dataset. The x-axis represents the ratio of the number of induced facts added to the number of facts in the original test graph, and the y-axis represents Hits@1. Across all datasets, an upward trend in Hits@1 is observed as the proportion of induced facts increases. However, increasing induced facts does not monotonically improve Hits@1, as seen in examples such as WN18RR-v3 (the green line on the left graph). Furthermore, computing time increases proportionately with induced facts, but Hits@1 does not, so there is a diminishing return.

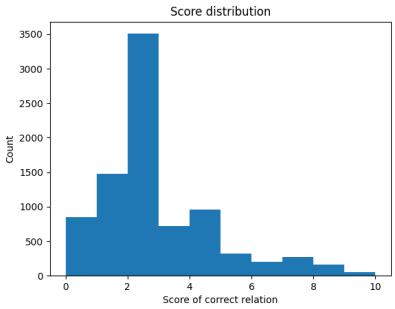
Despite these observations, the trends in the graphs provide reason to believe that further augmentation may be helpful, except in datasets where the gradient has plateaued, e.g. WN18RR-v4 (the red line on the left graph).

5.1.3 Inducing Fiercer Competition between Relations

In Section 3.2.2, it was proposed that rule paths based on induced facts are capable of producing high-confidence predictions due to strong competition. Figure 5.2 shows the score distribution of the correct relations on NELL-995 v4, the dataset which showed the most improvement with induced augmentation. The scores when no induced facts are used are mostly clustered between 0 and 1, making it hard to identify the correct relation. In contrast, the paths added by induced facts boost scores significantly; only around 27% of scores are lower than 2.



(a) Without induced facts



(b) With induced facts

Figure 5.2: Score distribution of NELL-995 v4

5.1.4 Limitations

The implementation of induced augmentation in this project assumes that all entities are related to each other, which is prone to generating noisy facts. However, the effect of these noisy facts is minimal (see Section 3.2.2).

The induced augmentation implementation is not scalable to large KG datasets with many entities, such as the original FB15k-237, which has 40,943 entities, and WN18RR, which has 14,541 entities. This is because the complexity is $O(E^2K)$, where E is the number of entities and K is the number of rules. Methods to overcome this limitation include inducing facts only when they are needed to complete a path, i.e. standard forward chaining [16], and randomly sampling tail-head combinations instead of iterating over all possible pairs.

5.2 KG Structure

The SOFIA model has a large performance advantage over TACT on FB15k-237 and NELL-995 as compared to WN18RR, where it falls behind.

One reason for this difference is that semantic correlations between relations are more complex in FB15k-237 and NELL-995 than in WN18RR [4], and SOFIA handles this complexity better than TACT. Relations in WN18RR are semantically strict lexical terms, and entities are concepts. In contrast, relations in FB15k-237 and NELL-995 can be used in multiple domains. For example, the relation *agentcontrols* in NELL-995

appears in triples which describe companies controlling websites, managers controlling companies, and sports leagues controlling sports teams.

Another possible explanation might be the structural properties of the KGs, namely, connectedness and diversity. The connectedness of a KG is measured by average node degree (number of incoming and outgoing edges) and *average cluster coefficient* (likelihood that a node's neighbours are connected). Entropy and *Gini coefficient* are used to measure its relational diversity. Entropy is defined as

$$Entropy = -\sum_{x \in X} p(x) \log_2 p(x) \tag{5.1}$$

where $p(x)$ is the proportion of triples in the KG that contain relation x, and average degree is defined as

$$avgDegree = 2 * \frac{\#Triples}{\#Entities} \tag{5.2}$$

The formulas for average cluster coefficient and Gini coefficient are not detailed here as they are relatively complicated and do not have statistically significant effects on the performance gain of SOFIA over TACT.

To identify which structural properties predict when SOFIA outperforms TACT and to what degree, univariate linear regression is conducted on #Relations, #Entities, #Triples, and the four attributes in Table 5.2. Of these seven factors, three have a statistically significant effect on the target variable $y = Hits@1_{SOFIA} - Hits@1_{TACT}$. However, entropy and #Relations are highly correlated ($r = 0.953$), so the interpretation is focused on entropy as it is a more comprehensive measure, accounting for both relation count and distribution.

The linear regression results support the hypothesis that both the connectedness (average degree) and relational diversity (entropy) of a KG are strong predictors of SOFIA's advantage over TACT. For each 1-unit increase in entropy, SOFIA gains an additional 7.3% Hits@1 over TACT, and for each 1-unit increase in average degree, SOFIA gains an additional 9.0% over TACT. Note that the coefficient of average degree being larger

	Avg Degree	Avg Cluster Coefficient	Entropy	Gini Coefficient
		WN18RR		
V1	4.30	0.093	1.30	0.757
V2	3.52	0.065	1.67	0.748
V3	2.93	0.029	2.63	0.557
V4	4.27	0.068	1.49	0.755
		FB15k-237		
V1	4.05	0.051	5.97	0.661
V2	5.47	0.077	6.10	0.685
V3	6.67	0.065	6.05	0.711
V4	8.66	0.081	6.22	0.700
		NELL-995		
V1	5.69	0.010	2.66	0.616
V2	5.14	0.082	4.77	0.714
V3	4.80	0.098	5.46	0.719
V4	5.79	0.065	3.99	0.759

Table 5.2: KG structure statistics

Factor	Coefficient	p-value	R^2
Entropy	7.332	0.0157	0.458
Avg Degree	8.960	0.0243	0.412
#Relations	0.17720	0.0282	0.396

Table 5.3: Linear effect of KG structure on Hits@1

than that of entropy does not mean that it is a better predictor of SOFIA's advantage. This happens because entropy and average degree are not scaled the same way.

The better predictor is identified using explained variance (R^2). Entropy explains 45.8% of the variance in the difference in performance between SOFIA and TACT, while average degree explains 41.2% of it. This means that the diversity and distribution of relations is slightly more impactful than the quantity of graph connections.

5.3 Rule Analysis

Some interesting examples of learned rules are given below.

WN18RR:

$$\begin{aligned} \texttt{verb\_group}(A,G) \text{ :-} \texttt{verb\_group}(A,B) \\ \wedge\, \texttt{hypernym}(B,C) \\ \wedge\, \texttt{derivationally\_related\_form}(C,D) \\ \wedge\, \texttt{has\_part}(D,E) \\ \wedge\, \texttt{derivationally\_related\_form}(E,F) \\ \wedge\, \texttt{hypernym}(F,G) \end{aligned} \tag{5.3}$$

FB15k-237:

$$\begin{aligned} \texttt{/location/country/capital}(A,D) \text{ :-} \\ \texttt{/military/combatants}(A,B) \\ \wedge\, \texttt{/film/film\_release\_region}(C,B) \\ \wedge\, \texttt{/film/film\_release\_region}(C,D) \end{aligned} \tag{5.4}$$

NELL-995:

$$\texttt{worksfor}(A,D) \text{ :- } \texttt{worksfor}(A,B), \texttt{worksfor}(C,B), \texttt{worksfor}(C,D) \tag{5.5}$$

Analysis is conducted on the characteristics of learned rules and their weights. This provides further insight into KG complexity (more rules are required to model more

complex KGs) and model confidence (rule weights are not probabilities, but can still be taken as measures of importance).

	Rules per Relation	Avg Rule Weight	% of Rules with Weight 1
		WN18RR	
V1	8.7	0.668	48.7
V2	8.9	0.713	51.7
V3	8.9	0.796	66.3
V4	8.4	0.792	63.2
		FB15k-237	
V1	3.2	0.938	89.2
V2	4.4	0.871	77.3
V3	5.4	0.847	72.9
V4	6.2	0.789	63.5
		NELL-995	
V1	10.8	0.896	84.8
V2	5.0	0.895	81.5
V3	5.8	0.897	82.1
V4	5.7	0.879	80.5

Table 5.4: Rule statistics

Since SOFIA is allowed to learn rules with a maximum length of 6 on WN18RR (compared to 4 on FB15k-237 and NELL-995), it is not surprising that it has the most rules per relation on average. The lower average weights of WN18RR rules reflect the ability of the SOFIA's linear program to tackle semantic distinctions by combining multiple rules to produce correct predictions with high confidence. FB15k-237 contains the most relations, which makes it hard to identify many reliable logical patterns. SOFIA handles this well by being more selective with the rules it learns and decreasing the mean rule weight when more rules have to be learnt, with a strong linear correlation ($r = -0.955$).

The rule statistics for NELL-995 are the most interesting. On average, it has the highest mean rule weight. In particular, NELL-995 v1 has the highest rules per relation and second highest mean rule weight among all datasets. A negative correlation ($r = -0.558$) is observed between rules per relation and mean rule weight. The correlation jumps to $r = -0.862$ when NELL-995 v1 is excluded. This suggests that

the rule set structure of NELL-995 v1 is a significant outlier, which is corroborated by its studentized residual of 4.08. The studentized residual of a data point is the error of a model fitted on all data excluding that point. Generally, data points with studentized residuals greater than 3 are classified as outliers.

This creates strong competition between relations in the induced augmentation process, which produces a set containing many high-quality induced facts. This anomaly is likely to be the main factor behind the unexpected performance gain of SOFIA+ on NELL-995 v1.

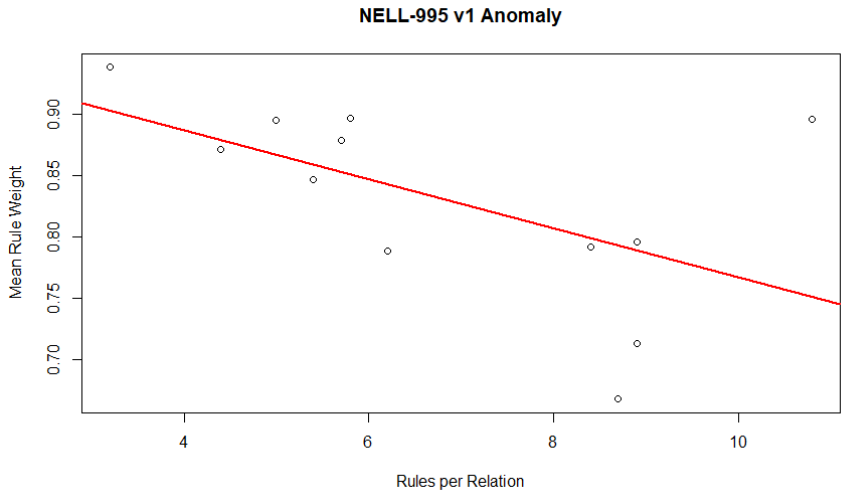


Figure 5.3: NELL-995 v1 anomaly in top right corner

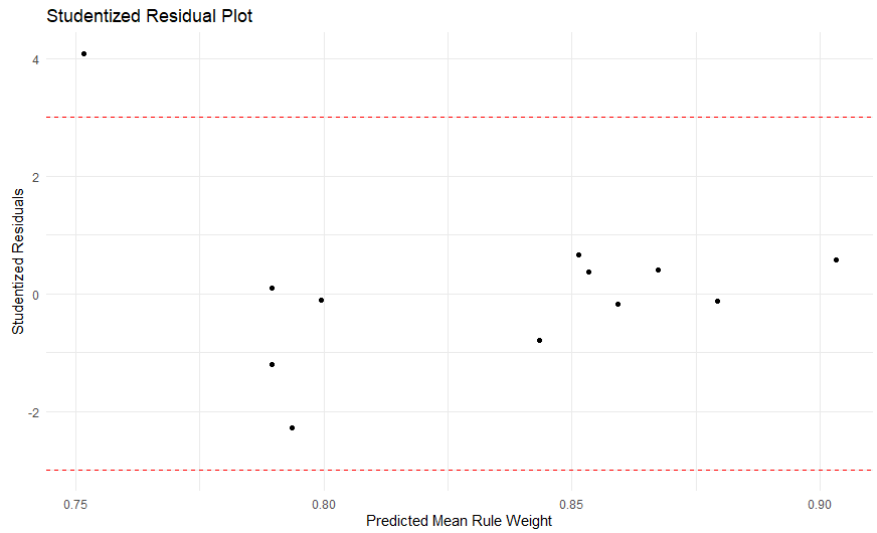


Figure 5.4: NELL-995 v1 anomaly in top left corner

5.4 Coverage and Discriminative Power

By definition, both induced augmentation and relaxed rules have a non-negative effect on coverage (see Definition 3), i.e. coverage never decreases under induced augmentation and relaxed rules because they make rules fire more often. However, blindly increasing coverage has the potential to increase noisy signals, which might end up overpowering the scores of the correct relations. Therefore, it is necessary to evaluate the discriminative power of the standard ruleset compared to that of the relaxed ruleset. Discriminative power is defined as

$$Discrimination = \frac{Hits@1}{Coverage} \tag{5.6}$$

	WN18RR				FB15k-237				NELL-995			
	V1	V2	V3	V4	V1	V2	V3	V4	V1	V2	V3	V4
Coverage												
SO	83.5	72.3	46.9	75.3	43.2	63.1	70.0	73.4	99.3	50.4	61.3	24.8
SOFIA	99.5	92.1	79.0	98.2	79.7	92.8	88.9	90.5	100	80.3	85.0	90.2
Discrimination												
SO	0.691	0.687	0.751	0.713	0.669	0.751	0.704	0.721	0.599	0.835	0.613	0.440
SOFIA	0.853	0.846	0.829	0.927	0.882	0.851	0.785	0.776	0.617	0.858	0.848	0.874

Table 5.5: Discriminative power

WN18RR				FB15k-237				NELL-995			
V1	V2	V3	V4	V1	V2	V3	V4	V1	V2	V3	V4
ΔCoverage											
16.0	19.8	32.1	22.9	36.5	29.7	18.9	17.1	0.7	29.9	23.7	65.4
ΔHits@1											
27.2	28.2	30.3	37.3	41.4	31.6	20.5	17.3	2.2	26.8	34.5	67.9

Table 5.6: Change in Hits@1 and coverage

From Table 5.5, it can be observed that the SOFIA model increases discriminative power across all datasets. Table 5.6 shows that on most datasets, the Hits@1 gained is greater than the coverage gained. This shows that the additional scores given by induced augmentation and relaxed rules did not just improve the model in the cases when the correct relation did not satisfy any full rules, but also helped the correct relation to achieve the highest score in cases where it already had some score but not enough to overpower another relation. To answer **RQ3** directly, the induced augmentation and relaxed rules components in the SOFIA model do not simply preserve the discriminative power of learned rules, but in fact increase discriminative power significantly.

5.5 Time Complexity

Let E be the number of entities, K be the number of rules learned, and T_{test} and T_{aug} be the respective numbers of triples in the test graph before and after the induced facts are added.

Training The simplex algorithm for solving LPs is superior in practice to the ellipsoid algorithm, which is proven to be polynomial in the worst case [24]. LPRules uses the column generation method [7], which is an extension of the simplex algorithm.

Induction and Inference The induced fact set generation process runs in $O(KE^2(E + T_{test}))$. Three components contribute to this time complexity: depth-first search $(E + T_{test})$ is performed for each rule (K) on each tail-head combination (E^2). The inference (or evaluation) of a single triple can be completed in $O(K(E + T_{aug}))$ time. Relaxed

	WN18RR(v1)	NELL-995(v1)	FB15k-237(v1)
TACT	0.22h	2.53h	8.97h
Training	30sec	48sec	32sec
Induction	17sec	16sec	121sec
Inference	711sec	336sec	39sec
SOFIA	758sec	400sec	192sec
Runtime reduction	4.3%	95.6%	99.4%

Table 5.7: Runtime of TACT vs enhanced LP model

rules do not increase computational complexity significantly, since checking the first and last hops in a path is performed by accessing the array of triples by index, which runs in $O(1)$.

Comparison The SOFIA runtime recorded in Table 5.7 demonstrates the sheer speed of linear programming as a rule-learning framework and shows how quickly rule-based models can perform inference.

Chapter 6

Conclusion and Future Work

SOFIA achieves state-of-the-art performance on FB15k-237 and NELL-995 datasets and shows that interpretability does not have to be sacrificed to maintain or improve predictive power. A substantial portion of SOFIA's predictive power lies in the innovation of the induced augmentation (IA) component, and the relaxed rule (F) component hardly contributes to performance. The results suggest that rule coverage should be increased by adding intermediate facts and paths via induced augmentation rather than accepting incomplete evidence with relaxed rules.

6.1 Future Work

The experiments were conducted with the induced fact set generated after only one iteration. Further work may investigate the effectiveness of multiple iterations and stricter selection methods to maintain the quality of facts induced in the later iterations.

A future study may also be done on the effects of induced augmentation on other rule-based models. Proving the effectiveness of induced augmentation in general will be the main goal of this study.

Induced augmentation solves the coverage problem faced by rule-based models. Future work should explore methods of increasing discriminative power.

Applying the models proposed in this project to large KGs such as the original WN18RR

and FB15k-237 datasets may also be interesting. Note that the lazy inference forward chaining method [16] is more efficient in larger datasets, as the number of potential induced facts is $\#entities^2$. This quadratic explosion can be reduced by dropping the assumption that all entities are related in some way, and developing methods to determine plausible tail-head combinations without checking all the rules on them.

The induced augmentation method may also contribute to the discovery of new relations between entities. Future projects might make note of the tail-head combinations which did not have any relations scoring at least 1, and use machine learning methods or manual annotation to label these with new relations.

The relaxed rule concept is simple and interesting but does not contribute to performance as expected. Some future work may involve adding more complexity and constraints to relaxed rules which can improve its contributions.

The refinement of chain-like rules into tree-like rules [25] is fascinating and clearly improves KGC model performance. However, experimenting with this idea was not possible due to insufficient memory and computing power.

Bibliography

[1] Yuxuan Lu, Shiyu Yang, and Benzhao Tang. "A Comprehensive Review of Relation Prediction Techniques in Knowledge Graph". In: *Web and Big Data. APWeb-WAIM 2023 International Workshops*. Ed. by Xiangyu Song et al. Singapore: Springer Nature Singapore, 2024, pp. 11–24. ISBN: 978-981-97-2991-3.

[2] Zhilin Yang, William W. Cohen, and Ruslan Salakhutdinov. *Revisiting Semi-Supervised Learning with Graph Embeddings*. 2016. arXiv: 1603.08861 [cs.LG]. URL: https://arxiv.org/abs/1603.08861.

[3] Komal K. Teru, Etienne Denis, and William L. Hamilton. "Inductive Relation Prediction by Subgraph Reasoning". In: *International Conference on Machine Learning*. 2020. URL: https://arxiv.org/abs/1911.06962.

[4] Jiajun Chen et al. *Topology-Aware Correlations Between Relations for Inductive Link Prediction in Knowledge Graphs*. 2021. arXiv: 2103.03642 [cs.LG]. URL: https://arxiv.org/abs/2103.03642.

[5] Alejandro Barredo Arrieta et al. *Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI*. 2019. arXiv: 1910.10045 [cs.AI]. URL: https://arxiv.org/abs/1910.10045.

[6] Zefan Zeng, Qing Cheng, and Yuehang Si. "Logical rule-based knowledge graph reasoning: A comprehensive survey". In: *Mathematics* 11.21 (2023), p. 4486.

[7] Sanjeeb Dash and João Goncalves. "Rule Induction in Knowledge Graphs Using Linear Programming". In: *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence*. Vol. 37. 4. 2023, pp. 4233–4241. DOI: 10.1609/aaai.v37i4.25541. URL: https://ojs.aaai.org/index.php/AAAI/article/view/25541.

[8] Antoine Bordes et al. “Translating embeddings for modeling multi-relational data”. In: vol. 26. 2013.

[9] Roberto Boselli, Simone D’Amico, and Navid Nobani. “eXplainable AI for word embeddings: a survey”. In: *Cognitive Computation* 17.1 (2025), p. 19.

[10] Ronky Francis Doh et al. “A systematic review of deep knowledge graph-based recommender systems, with focus on explainable embeddings”. In: *Data* 7.7 (2022), p. 94.

[11] Will Hamilton, Zhitao Ying, and Jure Leskovec. “Inductive representation learning on large graphs”. In: vol. 30. 2017.

[12] Liang Yao, Chengsheng Mao, and Yuan Luo. “KG-BERT: BERT for Knowledge Graph Completion”. In: *CoRR* abs/1909.03193 (2019). arXiv: 1909.03193. URL: http://arxiv.org/abs/1909.03193.

[13] Arjun R. Akula and Song-Chun Zhu. “Attention cannot be an Explanation”. In: *CoRR* abs/2201.11194 (2022). arXiv: 2201.11194. URL: https://arxiv.org/abs/2201.11194.

[14] Matthew Richardson and Pedro Domingos. “Markov logic networks”. In: *Machine learning* 62.1 (2006), pp. 107–136.

[15] Fan Yang, Zhilin Yang, and William W. Cohen. *Differentiable Learning of Logical Rules for Knowledge Base Reasoning*. 2017. arXiv: 1702.08367 [cs.AI]. URL: https://arxiv.org/abs/1702.08367.

[16] Kewei Cheng et al. “UniKER: A Unified Framework for Combining Embedding and Definite Horn Rule Reasoning for Knowledge Graph Inference”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Ed. by Marie-Francine Moens et al. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 9753–9771. DOI: 10.18653/v1/2021.emnlp-main.769. URL: https://aclanthology.org/2021.emnlp-main.769/.

[17] Amnon Shashua and Anat Levin. “Ranking with Large Margin Principle: Two Approaches”. In: *Advances in Neural Information Processing Systems*. Ed. by S. Becker, S. Thrun, and K. Obermayer. Vol. 15. MIT Press, 2002. URL:

https://proceedings.neurips.cc/paper_files/paper/2002/file/51de85ddd068f0bc787691d356176df9-Paper.pdf.

[18] Anthony Dixon and Adan E Vela. "Development of an Optimization-Based Ranking Framework for Concurrent Estimation of Student and Course Skills". In: *2025 IEEE Frontiers in Education Conference (FIE)*. IEEE. 2025, pp. 1–8.

[19] Fu-Yu Beverly Chen et al. "A Machine Learning with Ordinal Ranking Approach for Dysarthria Severity Classification in Parkinson's Disease". In: *Journal of Medical and Biological Engineering* (2025), pp. 1–12.

[20] Zhixiang Su, Di Wang, and Chunyan Miao. *Context Pooling: Query-specific Graph Pooling for Generic Inductive Link Prediction in Knowledge Graphs*. 2025. arXiv: 2507.07595 [cs.AI]. URL: https://arxiv.org/abs/2507.07595.

[21] Corinna Cortes and Vladimir Vapnik. "Support-vector networks". In: *Machine learning* 20.3 (1995), pp. 273–297.

[22] Lihui Liu, Zihao Wang, and Hanghang Tong. "Neural-symbolic reasoning over knowledge graphs: A survey from a query perspective". In: *ACM SIGKDD Explorations Newsletter* 27.1 (2025), pp. 124–136.

[23] Antoine Bordes et al. "Translating Embeddings for Modeling Multi-relational Data". In: *Advances in Neural Information Processing Systems*. Ed. by C.J. Burges et al. Vol. 26. Curran Associates, Inc., 2013. URL: https://proceedings.neurips.cc/paper_files/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf.

[24] Ron Shamir. "The efficiency of the simplex method: a survey". In: *Management science* 33.3 (1987), pp. 301–334.

[25] Wangtao Sun et al. *From Chain to Tree: Refining Chain-like Rules into Tree-like Rules on Knowledge Graphs*. 2025. arXiv: 2403.05130 [cs.AI]. URL: https://arxiv.org/abs/2403.05130.